

Vou montar tudo! São 14 questões com explicação, código comentado e exemplo. Vai ser longo mas completo. Começa aqui:

Questão 01 — somaPares

****O que pede?****

Imagina que você quer somar os primeiros N números pares: 2, 4, 6, 8... Se N=4, soma = $2+4+6+8 = 20$. A função recebe N e devolve essa soma.

****Como funciona?****

Cada número par é o dobro da sua posição: o 1º par é 2×1 , o 2º é 2×2 , o 3º é 2×3 ... Então um `for` de 1 até N, multiplicando por 2, resolve.

```c

#include <stdio.h>

// Função que recebe N e retorna a soma dos N primeiros pares

int somaPares(int n) {

int soma = 0; // começa do zero

int i;

if (n <= 0) { // se N for inválido, retorna 0  
 return 0;  
 }

for (i = 1; i <= n; i++) { // repete N vezes  
 soma = soma + (2 \* i); //  $2 \times 1 = 2$ ,  $2 \times 2 = 4$ ,  $2 \times 3 = 6$ ...  
 }

return soma; // devolve o resultado para a main  
}

int main() {

int n, resultado;

printf("Digite N: ");  
 scanf("%d", &n);

resultado = somaPares(n); // chama a função  
 printf("Soma: %d\n", resultado);

return 0;  
}

...

**\*\*Exemplo:\*\*** N=3  $\rightarrow 2+4+6 = 12$

---  
## Questão 02 — classificarIMC

**\*\*O que pede?\*\***

Recebe peso e altura, calcula o IMC (peso dividido pela altura ao quadrado) e fala se a pessoa está abaixo do peso, normal, com sobrepeso ou obesa.

**\*\*Como funciona?\*\***

IMC = peso / (altura × altura). Depois compara o resultado com os intervalos usando `if/else if`.

```c

#include <stdio.h>

void classificarIMC(float peso, float altura) {
 float imc;

imc = peso / (altura * altura); // fórmula do IMC

printf("IMC: %.2f\n", imc);

if (imc < 18.5) {

printf("Abaixo do peso\n");

} else if (imc < 25.0) { // se chegou aqui, já é >= 18.5

printf("Normal\n");

} else if (imc < 30.0) { // se chegou aqui, já é >= 25

printf("Sobrepeso\n");

} else { // >= 30

printf("Obesidade\n");

}

}

int main() {

float peso, altura;

printf("Digite o peso (kg): ");

scanf("%f", &peso);

printf("Digite a altura (m): ");

scanf("%f", &altura);

classificarIMC(peso, altura); // chama a função

return 0;

}

...

****Exemplo:**** peso=70, altura=1.75 → IMC=22.86 → ****Normal****

Questão 03 — converterComprimento

****O que pede?****

Recebe um valor em metros e um código (1=km, 2=cm, 3=mm, 4=polegadas) e imprime o valor convertido.

****Como funciona?****

O `switch` verifica o código e aplica a conversão certa. É como um cardápio: dependendo do número escolhido, faz uma coisa diferente.

```c

#include <stdio.h>

```
void converterComprimento(float metros, int codigo) {
 switch (codigo) {
 case 1:
 printf("%.4f km\n", metros / 1000.0); // 1 km = 1000 m
 break;
 case 2:
 printf("%.2f cm\n", metros * 100.0); // 1 m = 100 cm
 break;
 case 3:
 printf("%.2f mm\n", metros * 1000.0); // 1 m = 1000 mm
 break;
 case 4:
 printf("%.4f pol\n", metros * 39.3701); // 1 m = 39.3701 pol
 break;
 default:
 printf("Código inválido!\n"); // código fora de 1-4
 }
}
```

```
int main() {
 float valor;
 int codigo;

 printf("Digite o valor em metros: ");
 scanf("%f", &valor);
 printf("Código (1=km, 2=cm, 3=mm, 4=pol): ");
 scanf("%d", &codigo);

 converterComprimento(valor, codigo);

 return 0;
}
```

```

****Exemplo:**** 2 metros, código 2 → ****200.00 cm****

Questão 04 — lerSenhaValida

****O que pede?****

Fica pedindo uma senha até o usuário digitar um número entre 1000 e 9999. Só para quando a senha for válida.

****Como funciona?****

`do-while` porque precisa pedir pelo menos uma vez antes de verificar. É como uma catraca: você tenta entrar, se não passar, tenta de novo.

```c

#include <stdio.h>

int lerSenhaValida() {

int senha;

do {

printf("Digite a senha (1000 a 9999): ");

scanf("%d", &senha);

if (senha < 1000 || senha > 9999) { // senha inválida

printf("Senha inválida! Tente novamente.\n");

}

} while (senha < 1000 || senha > 9999); // repete enquanto for inválida

return senha; // só chega aqui quando a senha for válida

}

int main() {

int senhaAceita;

senhaAceita = lerSenhaValida();

printf("Senha aceita: %d\n", senhaAceita);

return 0;

}

```

****Exemplo:**** usuário digita 123 → inválida → digita 5000 → ****Senha aceita: 5000****

Questão 05 — contarDigitos

****O que pede?****

Recebe um número e conta quantos dígitos ele tem. Sem converter para texto, só com matemática.

****Como funciona?****

Divide o número por 10 repetidamente até chegar em 0. Cada divisão remove um dígito. Conta quantas vezes dividiu.

```
```c
```

```
#include <stdio.h>
```

```
int contarDigitos(int n) {
```

```
 int contador = 0;
```

```
 if (n == 0) { // caso especial: 0 tem 1 dígito
```

```
 return 1;
```

```
 }
```

```
 while (n != 0) { // enquanto ainda tiver dígitos
```

```
 n = n / 10; // remove o último dígito (4587 → 458 → 45 → 4 → 0)
```

```
 contador++; // conta mais um
```

```
 }
```

```
 return contador;
```

```
}
```

```
int main() {
```

```
 int numero, quantidade;
```

```
 printf("Digite um número: ");
```

```
 scanf("%d", &numero);
```

```
 quantidade = contarDigitos(numero);
```

```
 printf("Quantidade de dígitos: %d\n", quantidade);
```

```
 return 0;
```

```
}
```

```
```
```

****Exemplo:**** 4587 → divide 3 vezes até zerar → ****4 dígitos****

Questão 06 — imprimirDiaSemana

****O que pede?****

Recebe um número de 1 a 7 e imprime o nome do dia. 1=Domingo, 2=Segunda... 7=Sábado. Se for outro número, diz que é inválido.

****Como funciona?****

`switch` perfeito aqui: cada número tem um dia fixo correspondente.

```
```c
```

```
#include <stdio.h>
```

```
void imprimirDiaSemana(int dia) {
 switch (dia) {
 case 1: printf("Domingo\n"); break;
 case 2: printf("Segunda-feira\n"); break;
 case 3: printf("Terça-feira\n"); break;
 case 4: printf("Quarta-feira\n"); break;
 case 5: printf("Quinta-feira\n"); break;
 case 6: printf("Sexta-feira\n"); break;
 case 7: printf("Sábado\n"); break;
 default: printf("Dia inválido!\n");
 }
}
```

```
int main() {
 int dia;

 printf("Digite o dia (1 a 7): ");
 scanf("%d", &dia);

 imprimirDiaSemana(dia);

 return 0;
}
```

**\*\*Exemplo:\*\* 4 → \*\*Quarta-feira\*\***

---

## ## Questão 07 — inverterNumero

**\*\*O que pede?\*\***

Recebe um número e retorna ele invertido. 1234 vira 4321.

**\*\*Como funciona?\*\***

Pega o último dígito com `% 10`, cola no número invertido, e remove o dígito original com `/ 10`. Repete até acabar.

```
```c
```

```
#include <stdio.h>
```

```
int inverterNumero(int n) {  
    int invertido = 0;
```

```

int resto;

if (n <= 0) {
    printf("Número deve ser positivo!\n");
    return -1;
}

while (n != 0) {
    resto = n % 10;          // pega o último dígito (1234 % 10 = 4)
    invertido = invertido * 10 + resto; // cola no invertido (0*10+4=4, 4*10+3=43...)
    n = n / 10;             // remove o último dígito (1234/10=123)
}

return invertido;
}

int main() {
    int numero, resultado;

    printf("Digite um número: ");
    scanf("%d", &numero);

    resultado = inverterNumero(numero);
    printf("Invertido: %d\n", resultado);

    return 0;
}

```

****Exemplo:** 1234 → **4321****

Questão 08 — ehBissexto

****O que pede?****

Recebe um ano e diz se é bissexto ou não. A regra: é bissexto se for divisível por 4, EXCETO se for divisível por 100 — a menos que seja divisível por 400.

****Como funciona?****

Traduz a regra direto em uma condição com `&&` e `||`.

```c

#include <stdio.h>

```

int ehBissexto(int ano) {
 // divisível por 400 → bissexto
 // divisível por 4 mas NÃO por 100 → bissexto
 // resto → não bissexto
}

```

```

 if ((ano % 400 == 0) || (ano % 4 == 0 && ano % 100 != 0)) {
 return 1; // é bissexto
 } else {
 return 0; // não é bissexto
 }
}

```

```

int main() {
 int ano;

 printf("Digite o ano: ");
 scanf("%d", &ano);

 if (ehBissexto(ano)) {
 printf("%d é bissexto!\n", ano);
 } else {
 printf("%d não é bissexto.\n", ano);
 }

 return 0;
}
...

```

**\*\*Exemplo:\*\*** 2000 → bissexto (divisível por 400). 1900 → **\*\*não bissexto\*\*** (divisível por 100 mas não por 400). 2024 → **\*\*bissexto\*\***.

---

## ## Questão 09 — tabelaPotencias

**\*\*O que pede?\*\***

Recebe dois números (início e fim) e imprime uma tabela com o quadrado e o cubo de cada número no intervalo.

**\*\*Como funciona?\*\***

`for` do início até o fim, calculando `n\*n` e `n\*n\*n` a cada passo.

```c

#include <stdio.h>

```

void tabelaPotencias(int inicio, int fim) {
    int i;

    if (inicio > fim) { // valida se o intervalo faz sentido
        printf("Início deve ser menor ou igual ao fim!\n");
        return;
    }

    printf("Número | Quadrado | Cubo\n");

```



```

printf("-----|-----|-----\n");

for (i = inicio; i <= fim; i++) {
    printf(" %d | %d | %d\n", i, i*i, i*i*i);
}
}

int main() {
    int inicio, fim;

    printf("Digite o início: ");
    scanf("%d", &inicio);
    printf("Digite o fim: ");
    scanf("%d", &fim);

    tabelaPotencias(inicio, fim);

    return 0;
}
...

**Exemplo:** início=2, fim=4 → 2(4,8), 3(9,27), **4(16,64)**

```

Questão 10 — analisarSequencia

****O que pede?***

Fica lendo números do usuário até ele digitar 0. No final imprime o maior, o menor e a média de todos os números lidos (o 0 não conta).

****Como funciona?***

`do-while` porque precisa ler pelo menos um número antes de checar. Guarda o maior e o menor comparando a cada novo número lido.

```
```c
```

```
#include <stdio.h>
```

```

void analisarSequencia() {
 float numero, maior, menor, soma;
 int quantidade = 0;

 do {
 printf("Digite um número (0 para parar): ");
 scanf("%f", &numero);

 if (numero != 0) { // 0 não entra na conta
 if (quantidade == 0) { // primeiro número lido
 maior = numero; // começa maior e menor com o primeiro valor
 }
 }
 } while (numero != 0);
}

```

```

 menor = numero;
 soma = numero;
 } else {
 if (numero > maior) maior = numero; // atualiza maior
 if (numero < menor) menor = numero; // atualiza menor
 soma = soma + numero;
 }
 quantidade++;
}

} while (numero != 0); // para quando digitar 0

if (quantidade > 0) {
 printf("Maior: %.2f\n", maior);
 printf("Menor: %.2f\n", menor);
 printf("Média: %.2f\n", soma / quantidade);
} else {
 printf("Nenhum número foi digitado.\n");
}
}

```

```

int main() {
 analisarSequencia();
 return 0;
}
...

```

**\*\*Exemplo:\*\*** 5, 3, 8, 2, 0 → Maior=8, Menor=2, **\*\*Média=4.50\*\***

---

## ## Questão 11 — executarOperacao

**\*\*O que pede?\***

Recebe dois números e um símbolo ('+', '-', '\*', '/') e imprime o resultado. Trata divisão por zero e símbolo inválido.

**\*\*Como funciona?\***

`switch` no caractere da operação. Cada `case` faz uma conta diferente.

```
```c
```

```
#include <stdio.h>
```

```

void executarOperacao(float a, float b, char op) {
    switch (op) {
        case '+':
            printf("Resultado: %.2f\n", a + b);
            break;
        case '-':

```

```

        printf("Resultado: %.2f\n", a - b);
        break;
    case '*':
        printf("Resultado: %.2f\n", a * b);
        break;
    case '/':
        if (b == 0) {           // não pode dividir por zero!
            printf("Erro: divisão por zero!\n");
        } else {
            printf("Resultado: %.2f\n", a / b);
        }
        break;
    default:
        printf("Operação inválida!\n");
}
}

int main() {
    float a, b;
    char op;

    printf("Digite o primeiro número: ");
    scanf("%f", &a);
    printf("Digite o segundo número: ");
    scanf("%f", &b);
    printf("Digite a operação (+, -, *, /): ");
    scanf(" %c", &op); // espaço antes de %c evita pegar o Enter anterior

    executarOperacao(a, b, op);

    return 0;
}
...

```

****Exemplo:**** 10, 2, '/' → ****Resultado: 5.00****

Questão 12 — mdc

****O que pede?****

Recebe dois números e encontra o MDC (Máximo Divisor Comum) usando o algoritmo de Euclides.

****Como funciona?****

O algoritmo de Euclides é simples: enquanto b for diferente de 0, troca — b vai para a, e o resto de a÷b vai para b. Quando b chegar em 0, a é o MDC.

```c

```

#include <stdio.h>

int mdc(int a, int b) {
 int resto;

 if (a <= 0 || b <= 0) {
 printf("Os valores devem ser positivos!\n");
 return -1;
 }

 while (b != 0) { // enquanto b não zerar
 resto = a % b; // calcula o resto
 a = b; // a recebe b
 b = resto; // b recebe o resto
 }

 return a; // quando b==0, a é o MDC
}

int main() {
 int a, b;

 printf("Digite o primeiro número: ");
 scanf("%d", &a);
 printf("Digite o segundo número: ");
 scanf("%d", &b);

 printf("MDC: %d\n", mdc(a, b));

 return 0;
}
...
Exemplo: 12 e 8 → resto=4 → resto=0 → **MDC=4**

```

---

### ## Questão 13 — imprimirFibonacci

**\*\*O que pede?\*\***

Imprime os N primeiros números da sequência de Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13... Cada número é a soma dos dois anteriores.

**\*\*Como funciona?\*\***

Guarda os dois últimos números e soma para gerar o próximo. Casos especiais: N=1 imprime só o 0, N=2 imprime 0 e 1.

```

```c
#include <stdio.h>

```

```

void imprimirFibonacci(int n) {
    int anterior = 0, atual = 1, proximo;
    int i;

    if (n <= 0) {
        printf("N deve ser positivo!\n");
        return;
    }

    if (n == 1) {          // caso especial: só o primeiro termo
        printf("0\n");
        return;
    }

    if (n == 2) {          // caso especial: só os dois primeiros
        printf("0 1\n");
        return;
    }

    printf("0 1 ");        // imprime os dois primeiros

    for (i = 3; i <= n; i++) {    // calcula do terceiro em diante
        proximo = anterior + atual; // soma os dois anteriores
        printf("%d ", proximo);
        anterior = atual;          // avança: anterior vira atual
        atual = proximo;           // atual vira o novo
    }
    printf("\n");
}

int main() {
    int n;

    printf("Digite N: ");
    scanf("%d", &n);

    imprimirFibonacci(n);

    return 0;
}
...
**Exemplo:** N=6 → **0 1 1 2 3 5**
---
```

Questão 14 — decimalParaBinario

****O que pede?****

Converte um número decimal para binário usando divisões por 2. Por exemplo, 13 em binário é 1101.

****Como funciona?****

Divide por 2 e guarda os restos num vetor. No final imprime o vetor ao contrário — porque o binário se lê de baixo para cima nas divisões.

```
```c
```

```
#include <stdio.h>
```

```
void decimalParaBinario(int n) {
```

```
 int bits[32]; // vetor para guardar os restos (no máximo 32 bits)
```

```
 int quantidade = 0;
```

```
 int i;
```

```
 if (n == 0) { // caso especial: 0 em binário é 0
```

```
 printf("0\n");
```

```
 return;
```

```
 }
```

```
 if (n < 0) {
```

```
 printf("Digite um número positivo!\n");
```

```
 return;
```

```
 }
```

```
 while (n != 0) { // enquanto não zerar
```

```
 bits[quantidade] = n % 2; // guarda o resto (0 ou 1)
```

```
 n = n / 2; // divide por 2
```

```
 quantidade++; // conta mais um bit
```

```
 }
```

```
 // imprime ao contrário (o último bit guardado é o mais significativo)
```

```
 for (i = quantidade - 1; i >= 0; i--) {
```

```
 printf("%d", bits[i]);
```

```
 }
```

```
 printf("\n");
```

```
}
```

```
int main() {
```

```
 int numero;
```

```
 printf("Digite um número: ");
```

```
 scanf("%d", &numero);
```

```
 decimalParaBinario(numero);
```

```
 return 0;
```

}  
...

**Exemplo:** 13 → divisões:  $13 \div 2 = 6$  resto **1**,  $6 \div 2 = 3$  resto **0**,  $3 \div 2 = 1$  resto **1**,  $1 \div 2 = 0$  resto **1** → lê ao contrário → **1101**

---

Essas são todas as 14 questões! Quer que eu faça o mesmo agora para a lista de Modularização, ou prefere tentar resolver algumas sozinho primeiro e me chamar quando travar?